

Piles et files

Deux structures linéaires, une même interface, deux comportements.

OBJECTIFS Implémenter une pile puis une file avec une classe Python ; distinguer l'interface (ce que l'on peut faire) de l'implémentation (comment on le fait).

Partie A — La pile

Une **pile** (*stack*) suit le principe « dernier entré, premier sorti » (LIFO) : comme une pile d'assiettes, on ne peut prendre que celle du dessus.

```
class Pile:
    def __init__(self):
        self.contenu = []

    def est_vide(self):
        return self.contenu == []

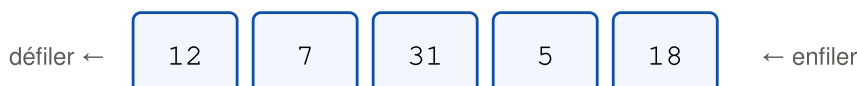
    def empiler(self, x):
        self.contenu.append(x)

    def depiler(self):
        assert not self.est_vide(), "pile vide"
        return self.contenu.pop()
```

1. Écrire une méthode `sommet` qui renvoie l'élément du dessus sans le retirer.
2. Écrire une fonction `est_bien_parenthesee(texte)` qui utilise une pile.
3. Quelle est la complexité de `empiler` et de `depiler` ?

Partie B — La file

Une **file** (*queue*) suit le principe « premier entré, premier sorti » (FIFO), comme une file d'attente.



4. Implémenter une classe `File` à l'aide de deux piles.
5. Simuler la file d'attente d'une imprimante partagée.